Dr. D. Y. Patil
Founder, Dr. D. Y. Patil Group

Dr. Sushant Patil
President, DYPEF

Dr. Suresh Mali
Principal, DYPCOEI

Unit II- Linear Data Structures, searching and sorting

Overview of Array, Array as an Abstract Data Type, Operations on Array, Storage Representation, Multidimensional Arrays[2D, nD], Sparse matrix representation using 2D Searching: Sequential Search/Linear Search, Binary Search, Fibonacci Search, and Indexed Sequential Search.

Sorting: Concepts- Stability, Efficiency, and Number of Passes, Internal and External Sorting, Bubble sort, Insertion Sort, Selection Sort , Quick Sort, Merge sort

Overview of Array,

An array is a linear data structure that stores a fixed-size sequence of elements of the same data type in contiguous memory locations. Each element in an array is identified by an index, starting from 0. Arrays allow efficient access to elements using these indices, which makes them ideal for storing and manipulating lists of data such as numbers, strings, or objects. Arrays support operations like traversal, insertion, deletion, searching, and sorting, although insertion and deletion at arbitrary positions can be costly due to shifting elements. Arrays are widely used due to their simplicity and fast access time, especially when the size of the data is known in advance. However, they lack flexibility in dynamic memory usage, which is where structures like linked lists are preferred.

Array as an Abstract Data Type,

An array as an Abstract Data Type (ADT) is a logical model that defines a collection of elements stored in a fixed-size sequential structure, where each element is identified by a unique index. As an ADT, the array focuses on the operations that can be performed—such as accessing, inserting, deleting, or updating elements—without concerning itself with the underlying implementation. The key characteristics of the array ADT include constant-time access to elements using the index (random access) and contiguous memory allocation. Although arrays have fixed sizes once created, they provide efficient and predictable performance for accessing and iterating through data. This makes them ideal for use cases like lookup tables, matrices, and buffers in various applications.

Arrays support several fundamental operations that are commonly used in programming.

TY_SPPU_FE_ENGG

Click here join Telegram channel

PDF BY VL







✨ WHY JOIN ?

🏆 Trusted by 80,000+ Engineering Students
🔔 Stay Updated with latest university news
🚀 Make your engineering journey easy with us

Best Regards, Vishwajeet Londhe

© 2026 Vishwajeet Londhe. All Rights Reserved.

The primary operation is traversal, where each element of the array is accessed and processed one by one.

Insertion refers to adding a new element at a specific index, which may involve shifting other elements to maintain order—this can be time-consuming in large arrays.

Deletion involves removing an element from a given index, which also typically requires shifting elements to fill the gap. S

earching is used to find the position of a particular value in the array, using methods like linear or binary search.

Updating or modifying an element is straightforward since arrays allow direct access using an index. These operations make arrays a versatile and efficient structure for managing fixed-size collections of data.

Storage Representation

The storage representation of arrays refers to how array elements are stored in contiguous memory locations in a computer's memory. Each element in an array is of the same data type and occupies a fixed amount of space. Because of the contiguous layout, the memory address of any element can be calculated directly using the formula:

Address of A[i] = Base Address + (i × Size of each element)

This direct addressing allows for constant-time (O(1)) access to any element. In multi-dimensional arrays like 2D or 3D arrays, elements are also stored in a single block of memory, typically in row-major or column-major order depending on the language. This representation ensures efficient indexing but makes insertion and deletion at arbitrary positions costly due to shifting of elements.

Multidimensional Arrays[2D, nD],

A **multi-dimensional array in C** can be defined as an array that has more than one dimension. Having more than one dimension means that it can grow in multiple directions. Some popular multidimensional arrays include 2D arrays which grows in two dimensions, and 3D arrays which grows in three dimensions.

Syntax

The general form of declaring **N-dimensional arrays** is shown below:

```
type arrName[size1][size2]....[sizeN];
```

- **type**: Type of data to be stored in the array.
- **arrName**: Name assigned to the array.
- **size1, size2,..., sizeN**: Size of each dimension.

Types of Multidimensional Arrays

In C, there can be many types of arrays depending on their dimensions but two of them are most commonly used:

TG:SPPU_FE_ENGG

Click here join Telegram channel

1. 2D Array - Two Dimensional
2. 3D Array - Three Dimensional

2D Arrays in C

A **two-dimensional array** or **2D array** is the simplest form of the multidimensional array. We can visualize a two-dimensional array as one-dimensional arrays stacked vertically

forming a table with 'm' rows and 'n' columns. In C, arrays are 0-indexed, so the row number ranges from 0 to (m-1) and the column number ranges from 0 to (n-1).

What is a sparse matrix?



Sparse matrices are those matrices that have the majority of their elements equal to zero. In other words, the sparse matrix can be defined as the matrix that has a greater number of zero elements than the non-zero elements.

Now, the question arises: we can also use the simple matrix to store the elements, then why is the sparse matrix required?

Why is a sparse matrix required if we can use the simple matrix to store elements?
There are the following benefits of using the sparse matrix -

Storage - We know that a sparse matrix contains lesser non-zero elements than zero, so less memory can be used to store elements. It evaluates only the non-zero elements.

Computing time: In the case of searching in sparse matrix, we need to traverse only the non-zero elements rather than traversing all the sparse matrix elements. It saves computing time by logically designing a data structure traversing non-zero elements.

Representation of sparse matrix

Now, let's see the representation of the sparse matrix. The non-zero elements in the sparse matrix can be stored using triplets that are rows, columns, and values. There are two ways to represent the sparse matrix that are listed as follows -

- Array representation
- Linked list representation

Array representation of the sparse matrix

Representing a sparse matrix by a 2D array leads to the wastage of lots of memory. This is because zeroes in the matrix are of no use, so storing zeroes with non-zero elements is wastage of memory. To avoid such wastage, we can store only non-zero elements. If we store only non-zero elements, it reduces the traversal time and the storage space.

TG:SPPU_FE_ENGG

Click here join Telegram channel

In 2D array representation of sparse matrix, there are three fields used that are named as -

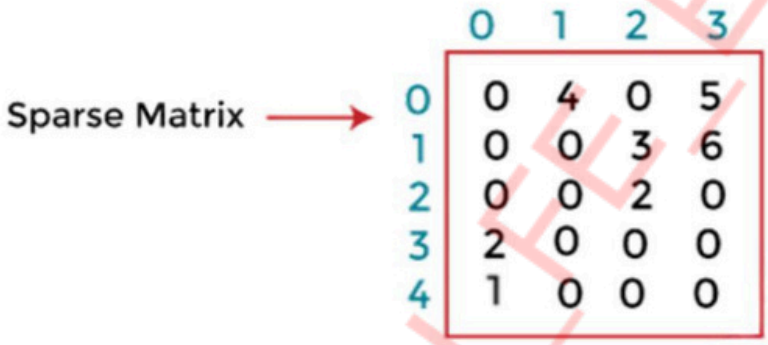
ROW	COL	VALUE

- **Row -** It is the index of a row where a non-zero element is located in the matrix.
- **Column -** It is the index of the column where a non-zero element is located in the matrix.
- **Value -** It is the value of the non-zero element that is located at the index (row, column).

Example -

Let's understand the array representation of sparse matrix with the help of the example given below -

Consider the sparse matrix -



In the above figure, we can observe a 5x4 sparse matrix containing 7 non-zero elements and 13 zero elements. The above matrix occupies 5x4 = 20 memory space. Increasing the size of matrix will increase the wastage space.

The tabular representation of the above matrix is given below -

TG:SPPU_FE_ENGG

Click here join Telegram channel

Table Structure

Row	Column	Value
0	1	4
0	3	5
1	2	3
1	3	6
2	2	2
3	0	2
4	0	1
5	4	7

In the above structure, first column represents the rows, the second column represents the columns, and the third column represents the non-zero value. The first row of the table represents the triplets. The first triplet represents that the value 4 is stored at 0th row and 1st column. Similarly, the second triplet represents that the value 5 is stored at the 0th row and 3rd column. In a similar manner, all triplets represent the stored location of the non-zero elements in the matrix.

The size of the table depends upon the total number of non-zero elements in the given sparse matrix. Above table occupies 8x3 = 24 memory space which is more than the space occupied by the sparse matrix. So, what's the benefit of using the sparse matrix? Consider the case if the matrix is 8*8 and there are only 8 non-zero elements in the matrix, then the space occupied by the sparse matrix would be 8*8 = 64, whereas the space occupied by the table represented using triplets would be 8*3 = 24.

Linear Search Algorithm

Suppose we are searching a target element in an array. In linear search we begin with the first position of the array, and traverse the whole array in order to find the target element. If we find the target element we return the index of the element. Otherwise, we will move to the next position. If we arrive at the last position of an array and still can not find the target, we return -1. This is called the Linear search or Sequential search.

Time Complexity: O(n), where n is the size of the input array. The worst-case scenario is when the target element is not present in the array, and the function has to go through the entire array to figure that out.

TG SPPU_FE_ENGG

Click here join Telegram channel

Auxiliary Space: O(1), the function uses only a constant amount of extra space to store variables. The amount of extra space used does not depend on the size of the input array.

BINARY SEARCH

Binary Search is a more optimized form of searching algorithm. It cuts down the search space in halves achieving logarithmic time complexity on a sorted data. We take two extremes lower bound and upper bound and compare our target element with the middle element. In the process we discard one half where we are sure our target element can not be found and update our lower and upper bound accordingly.

Time Complexity: O(log n) - Binary search algorithm divides the input array in half at every step, reducing the search space by half, and hence has a time complexity of logarithmic order.

Auxiliary Space: O(1) - Binary search algorithm requires only constant space for storing the low, high, and mid indices, and does not require any additional data structures, so its auxiliary space complexity is O(1).

Linear Search	Binary Search
In linear search input data need not to be in sorted.	In binary search input data need to be in sorted order.
It is also called sequential search.	It is also called half-interval search.
The time complexity of linear search **O(n)**.	The time complexity of binary search **O(log n)**.
Multidimensional array can be used.	Only single dimensional array is used.
Linear search performs equality comparisons.	Binary search performs ordering comparisons.

TG:SPPU_FE_ENGG

Click here join Telegram channel

It is less complex.	It is more complex.
It is very slow process.	It is very fast process.

Fibonacci Search Algorithm

The Fibonacci Search Algorithm makes use of the Fibonacci Series to diminish the range of an array on which the searching is set to be performed. With every iteration, the search range decreases making it easier to locate the element in the array. The detailed procedure of the searching is seen below −

Step 1 − As the first step, find the immediate Fibonacci number that is greater than or equal to the size of the input array. Then, also hold the two preceding numbers of the selected Fibonacci number, that is, we hold Fm, Fm-1, Fm-2 numbers from the Fibonacci Series.

Step 2 − Initialize the offset value as -1, as we are considering the entire array as the searching range in the beginning.

Step 3 − Until Fm-2 is greater than 0, we perform the following steps −

Compare the key element to be found with the element at index [min(offset+Fm-2,n-1)]. If a match is found, return the index.

If the key element is found to be lesser value than this element, we reduce the range of the input from 0 to the index of this element. The Fibonacci numbers are also updated with Fm = Fm-2.

But if the key element is greater than the element at this index, we remove the elements before this element from the search range. The Fibonacci numbers are updated as Fm = Fm-1. The offset value is set to the index of this element.

Step 4 − As there are two 1s in the Fibonacci series, there arises a case where your two preceding numbers will become 1. So if Fm-1 becomes 1, there is only one

TG-SPPU_FE_ENGG

Click here join Telegram channel

element left in the array to be searched. We compare the key element with that element and return the 1st index. Otherwise, the algorithm returns an unsuccessful search.

Indexed Sequential Search

In this searching method, first of all, an index file is created, that contains some specific group or division of required record when the index is obtained, then the partial indexing takes less time cause it is located in a specified group.

Characteristics of Indexed Sequential Search:

- In Indexed Sequential Search a sorted index is set aside in addition to the array.
- Each element in the index points to a block of elements in the array or another expanded index.
- The index is searched 1st then the array and guides the search in the array.

1. Stability in Sorting

Definition: A sorting algorithm is stable if it preserves the relative order of equal elements.

Example:

If you sort the following list by the first number only:

[(4, 'A'), (3, 'B'), (4, 'C')]

A stable sort will give: [(3, 'B'), (4, 'A'), (4, 'C')]

An unstable sort might give: [(3, 'B'), (4, 'C'), (4, 'A')]

Importance:

Useful when multiple passes are done on different keys (e.g., sort by name, then by age).

Stability is often preferred in database and record systems.

Stable Algorithms: Bubble Sort, Insertion Sort, Merge Sort, Counting Sort

Unstable Algorithms: Selection Sort, Quick Sort, Heap Sort

3. Number of Passes

Definition: A *pass* is a complete iteration over the array where comparisons/swaps are made.

- **Bubble Sort**: May require up to *n-1* passes (worst case).
- **Selection Sort**: Always makes *n-1* passes.
- **Insertion Sort**: Up to *n-1* passes.

TG:SPPU_FE_ENGG

Click here join Telegram channel

- **Merge Sort**: $\log_2 n$ passes (recursive divide steps).
- **Quick Sort**: $\log_2 n$ average passes (depends on pivot).
- **Counting Sort**: 1 or 2 passes depending on implementation.

Impact:

- Fewer passes usually mean better performance.
- In-place sorts (like Quick Sort, Heap Sort) avoid extra memory usage per pass.

Internal Sorting

Definition: Sorting where the **entire dataset fits into the main memory (RAM)**.

Used when: The number of records is small enough to be loaded fully into memory.

Examples:

- Bubble Sort
- Insertion Sort
- Quick Sort
- Merge Sort (in-memory)

Features:

- Fast due to direct memory access.
- Simpler to implement.
- Common in standard applications, databases, and systems with manageable data sizes

External Sorting

Definition: Sorting large volumes of data that **cannot fit into RAM**; instead, data is stored and processed in **secondary storage** (like hard disks).

Used when: The dataset is too large to load all at once (e.g., big logs, multimedia databases).

Example:

- **External Merge Sort** (most common)

Working Principle:

1. Divide large data into **chunks** that fit into memory.
2. Sort each chunk using internal sorting.
3. Merge sorted chunks using efficient **multi-way merge**.

Features:

- Optimized for **minimizing disk I/O**.
- Used in big data systems, database indexing, and file systems.

Bubble sort

TG:SPPU_FE_ENGG

Click here join Telegram channel

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity are quite high.

- We sort the array using multiple passes. After the first pass, the maximum element goes to end (its correct position). Same way, after second pass, the second largest element goes to second last position and so on.
- In every pass, we process only those elements that have already not moved to correct position. After k passes, the largest k elements must have been moved to the last k positions.
- In a pass, we consider remaining elements and compare all adjacent and swap if larger element is before a smaller element. If we keep doing this, we get the largest (among the remaining elements) at its correct position.

Step 1: We start with an unsorted array.

```
[7, 12, 9, 11, 3]
```

Step 2: We look at the two first values. Does the lowest value come first? Yes, so we don't need to swap them.

```
[7, 12, 9, 11, 3]
```

Step 3: Take one step forward and look at values 12 and 9. Does the lowest value come first? No.

```
[7, 12, 9, 11, 3]
```

Step 4: So we need to swap them so that 9 comes first.

```
[7, 9, 12, 11, 3]
```

Step 5: Taking one step forward, looking at 12 and 11.

```
[7, 9, 12, 11, 3]
```

Step 6: We must swap so that 11 comes before 12.

```
[7, 9, 11, 12, 3]
```

TG:SPPU_FE_ENGG

Click here join Telegram channel

Step 7: Looking at 12 and 3, do we need to swap them? Yes.

```
[7, 9, 11, 12, 3]
```

Step 8: Swapping 12 and 3 so that 3 comes first.

```
[7, 9, 11, 3, 12]
```

Insertion Sort

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. It is like sorting playing cards in your hands. You split the cards into two groups: the sorted cards and the unsorted cards. Then, you pick a card from the unsorted group and put it in the right place in the sorted group.

- We start with the second element of the array as the first element is assumed to be sorted.
- Compare the second element with the first element if the second element is smaller then swap them.
- Move to the third element, compare it with the first two elements, and put it in its correct position
- Repeat until the entire array is sorted.

Step 1: We start with an unsorted array.

```
[ 7, 12, 9, 11, 3]
```

Step 2: We can consider the first value as the initial sorted part of the array. If it is just one value, it must be sorted, right?

```
[ 7, 12, 9, 11, 3]
```

Step 3: The next value 12 should now be moved into the correct position in the sorted part of the array. But 12 is higher than 7, so it is already in the correct position.

```
[ 7, 12, 9, 11, 3]
```

Step 4: Consider the next value 9.

```
[ 7, 12, 9, 11, 3]
```

TG:SPPU_FE_ENGG

Click here join Telegram channel

Step 5: The value 9 must now be moved into the correct position inside the sorted part of the array, so we move 9 in between 7 and 12.

```
[ 7, 9, 12, 11, 3]
```

Step 6: The next value is 11.

```
[ 7, 9, 12, > 11, 3]
```

Step 7: We move it in between 9 and 12 in the sorted part of the array.

```
[ 7, 9, 11, 12, 3]
```

Step 8: The last value to insert into the correct position is 3.

```
[ 7, 9, 11, 12, 3]
```

Step 9: We insert 3 in front of all other values because it is the lowest value.

```
[ 3,7, 9, 11, 12]
```

Selection Sort

Selection Sort is a comparison-based sorting algorithm. It sorts an array by repeatedly selecting the **smallest (or largest)** element from the unsorted portion and swapping it with the first unsorted element. This process continues until the entire array is sorted.

1. First we find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.

2. Then we find the smallest among remaining elements (or second smallest) and swap it with the second element.
3. We keep doing this until we get all elements moved to correct position.

Step 1: We start with an unsorted array.

```
[ 7, 12, 9, 11, 3]
```

TG:SPPU_FE_ENGG

Click here join Telegram channel

Step 2: Go through the array, one value at a time. Which value is the lowest? 3, right?

```
[ 7, 12, 9, 11, 3]
```

Step 3: Move the lowest value 3 to the front of the array.

```
[ 3, 7, 12, 9, 11]
```

Step 4: Look through the rest of the values, starting with 7. 7 is the lowest value, and already at the front of the array, so we don't need to move it.

```
[ 3, 7, 12, 9, 11]
```

Step 5: Look through the rest of the array: 12, 9 and 11. 9 is the lowest value.

```
[ 3, 7, 12, 9, 11]
```

Step 6: Move 9 to the front.

```
[ 3, 7, 9, 12, 11]
```

Step 7: Looking at 12 and 11, 11 is the lowest.

```
[ 3, 7, 9, 12, 11]
```

Step 8: Move it to the front.

```
[ 3, 7, 9, 11, 12]
```

Quick Sort

QuickSort is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.
It works on the principle of **divide and conquer**, breaking down the problem into smaller sub-problems.
There are mainly three steps in the algorithm:

TG:SPPU_FE_ENGG

Click here join Telegram channel

1. **Choose a Pivot:** Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element, or median).
2. **Partition the Array:** Rearrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right. The pivot is then in its correct position, and we obtain the index of the pivot.
3. **Recursively Call:** Recursively apply the same process to the two partitioned sub-arrays (left and right of the pivot).
4. **Base Case:** The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.

5. **Step 1:** We start with an unsorted array.
6. `[ 11, 9, 12, 7, 3]`
7. ---
8. **Step 2:** We choose the last value 3 as the pivot element.
9. `[ 11, 9, 12, 7, 3]`
10. ---
11. **Step 3:** The rest of the values in the array are all greater than 3, and must be on the right side of 3. Swap 3 with 11.
12. `[ 3, 9, 12, 7, 11]`
13. ---
14. **Step 4:** Value 3 is now in the correct position. We need to sort the values to the right of 3. We choose the last value 11 as the new pivot element.
15. `[ 3, 9, 12, 7, 11]`
16. ---
17. **Step 5:** The value 7 must be to the left of pivot value 11, and 12 must be to the right of it. Move 7 and 12.
18. `[ 3, 9, 7, 12, 11]`
19. ---
20. **Step 6:** Swap 11 with 12 so that lower values 9 and 7 are on the left side of 11, and 12 is on the right side.
21. `[ 3, 9, 7, 11, 12]`
22. ---
23. **Step 7:** 11 and 12 are in the correct positions. We choose 7 as the pivot element in sub-array [9, 7], to the left of 11.
24. `[ 3, 9, 7, 11, 12]`
25. ---
26. **Step 8:** We must swap 9 with 7.
27. `[ 3, 7, 9, 11, 12]`

TG:SPU FE_ENGG

Click here join Telegram channel



CLICK HERE TO JOIN WHATSAPP CHANNEL FOR SPPU UNIVERSITY

👇 **JUST CLICK ON LINK** 👇

https://www.whatsapp.com/channel/0029ValjFrilCVfpcV9HFc3b

TGSPPU_FE_ENGG